P R O D U C T R E Q U I R E M E N T S D O C U M E N T

StreamTogether

Anime & Movies Streaming Portal with Watch Party

Version 1.0 • March 2026 • Confidential

Stack: React + Node.js + Firebase Auth + Firestore + Cloud Run (min-instances 0)

Content: Consumet API + AniList + TMDB + Real-Debrid

Table of Contents

1. Product Overview & Goals
2. Legal Architecture
3. Tech Stack & Architecture
4. Project File Structure
5. Environment Variables
6. External Services Setup
7. Database Schema (Firestore)
8. Content APIs
   8.1 Consumet API (Anime Streams)
   8.2 AniList API (Anime Metadata)
   8.3 TMDB API (Movies & TV Metadata)
   8.4 Real-Debrid API (Movie/TV Streams)
9. Feature Specifications
   9.1 Authentication
   9.2 Home / Discovery Page
   9.3 Anime Browse & Search
   9.4 Movies & TV Browse & Search
   9.5 Content Detail Page
   9.6 Watch Page (Player + Sync + Chat)
   9.7 Watch Party Rooms
   9.8 User Profile & Watchlist
10. API Endpoints (Backend)
11. WebSocket Events & Reconnect Strategy
12. UI/UX Design Specifications
13. Page-by-Page Component Specs
14. Deployment (Cloud Run)
15. Cold Start & Reconnect Handling
16. Firestore Security Rules
17. Error Handling & Edge Cases
18. Acceptance Criteria

1. Product Overview & Goals

StreamTogether is a self-hosted web streaming portal that lets authenticated users browse anime and movies/TV shows, stream them directly in the browser, and watch together in synchronized rooms with live chat. It is a combination of a 9anime-style anime portal and a Stremio-style movies/TV portal, built entirely with free-tier or self-hosted services and deployed on Google Cloud Run.

> **One-line pitch**
> A private Netflix + Crunchyroll alternative: browse anime and movies, hit play, invite friends, and watch in sync — all in the browser, hosted for nearly $0.

1.1 Goals

- Users can browse, search, and stream anime episodes via Consumet API (self-hosted).
- Users can browse, search, and stream movies and TV shows via Real-Debrid (user's own API key).
- Content metadata (posters, descriptions, ratings, episode lists) comes from AniList (anime) and TMDB (movies/TV).
- Users can create watch party rooms, invite friends via a code, and watch in sync.
- Playback sync (play/pause/seek) over WebSockets with automatic reconnect on Cloud Run cold starts.
- Live chat inside each watch room, stored in Firestore.
- Users maintain a personal watchlist and continue-watching history.
- Entire app runs on Cloud Run with min-instances=0 (free tier) — WebSocket reconnect logic compensates for cold starts.

1.2 Non-Goals (v1)

- No downloading or offline playback.
- No user-uploaded content.
- No comments/reviews on content.
- No transcoding or media proxying — video streams directly from source to browser.
- No mobile app — web only (but must be mobile-responsive).
- No admin panel in v1.

2. Legal Architecture

This section explains the legal design decisions. The coding agent must understand why the system is built this way.

2.1 Consumet API — Self-Hosted

Consumet is an open-source Node.js library/API that aggregates anime stream sources. The developer (you) self-hosts their own Consumet instance. Consumet itself does not store or redistribute content — it resolves stream URLs on demand from public sources. You run it as a separate Cloud Run service.

> **IMPORTANT for coding agent**
> Consumet API is a SEPARATE Cloud Run service from the main app. Deploy it from https://github.com/consumet/api.consumet.org using its own Dockerfile. The main app calls it as an internal API. Do NOT bundle Consumet into the main app.

2.2 Real-Debrid — User's Own Key

Real-Debrid is a premium link resolver service. Each user provides their OWN Real-Debrid API key in their profile settings. The app uses their key to resolve magnet links or torrent hashes into direct stream URLs. The app never stores torrent hashes or magnet links — the user's browser sends them to Real-Debrid directly. Legal exposure is entirely on the individual user, not the platform operator. This is the exact model Stremio uses.

2.3 AniList & TMDB — Metadata Only

AniList and TMDB are used only for metadata: titles, descriptions, poster images, ratings, episode lists, cast. No video content comes from these APIs. Both are free and explicitly allow third-party app usage with API keys.

3. Tech Stack & Architecture

3.1 Technology Decisions

Layer	Technology	Why
Frontend	React 18 + Vite	Fast, easy static deployment
Styling	Tailwind CSS v3	Utility-first, dark theme friendly
Video Player	Video.js 8 + HLS.js	HLS streams, MP4, subtitle support
Real-time sync	Socket.IO v4	WebSocket + polling fallback, auto-reconnect
Auth	Firebase Auth	Email + Google OAuth, free up to 50k MAU
Database	Firestore	Watchlist, history, rooms, chat — real-time listeners
Backend API	Node.js 20 + Express 4	Thin proxy + auth + room management
Anime streams	Consumet API (self-hosted)	Open-source anime stream aggregator
Anime metadata	AniList GraphQL API	Free, no key required, comprehensive
Movie/TV metadata	TMDB REST API	Free with API key, industry standard
Movie/TV streams	Real-Debrid API (user key)	Premium link resolver, user's own account
Container	Docker node:20-alpine	Single image for main app
Hosting	Google Cloud Run	min-instances=0, scales to zero, free tier
Consumet hosting	Google Cloud Run (separate)	Second Cloud Run service, also free tier
DNS/Proxy	Cloudflare	Free SSL, DDoS protection

3.2 Architecture Overview

```
Browser (React SPA)
  |-- REST --> Main Backend (/api/*)          [Express, Cloud Run service #1]
  |-- WebSocket --> Socket.IO                 [same process as main backend]
  |-- REST --> Consumet API                   [Cloud Run service #2, anime
streams]
  |-- GraphQL --> AniList API                 [external, free, no auth]
```

```
  |-- REST --> TMDB API                        [external, free API key]
  |-- REST --> Real-Debrid API                 [external, user's own key]
  |-- Firestore SDK (direct) --> Firestore     [watchlist, history, rooms,
chat]
  |-- Firebase Auth SDK --> Firebase Auth      [login, token]
```

Cold start strategy
Main backend: min-instances=0 (free). On cold start (~3-5s), Socket.IO client retries with exponential backoff. The frontend shows a 'Connecting...' overlay and automatically reconnects. Consumet: also min-instances=0. Anime stream requests show a loading state while Consumet wakes up.

4. Project File Structure

Create EXACTLY this structure. Every file listed must exist.

```
streamtogether/                                <- repo root
├── Dockerfile                                 <- main app only
├── .dockerignore
├── .gitignore
├── README.md
│
├── frontend/                                  <- React SPA
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── postcss.config.js
│   ├── index.html
│   ├── .env.example
│   └── src/
│       ├── main.jsx
│       ├── App.jsx                            <- React Router setup
│       ├── firebase.js                        <- Firebase init
│       ├── hooks/
│       │   ├── useAuth.js                     <- AuthContext, useAuth,
useAuthFetch
│       │   ├── useSocket.js                   <- Socket.IO with reconnect
logic
│       │   ├── useWatchlist.js                <- Firestore watchlist CRUD
│       │   └── useHistory.js                  <- Firestore watch history
│       ├── pages/
│       │   ├── AuthPage.jsx
│       │   ├── HomePage.jsx                   <- trending anime + movies
│       │   ├── AnimeBrowsePage.jsx            <- browse/search anime
│       │   ├── AnimeDetailPage.jsx            <- anime info + episode list
│       │   ├── MovieBrowsePage.jsx            <- browse/search movies+TV
│       │   ├── MovieDetailPage.jsx            <- movie/TV info + seasons
│       │   ├── WatchPage.jsx                  <- video player + chat + sync
│       │   ├── RoomsPage.jsx                  <- list + create watch rooms
│       │   └── ProfilePage.jsx                <- watchlist, history, RD key
│       ├── components/
│       │   ├── ProtectedRoute.jsx
│       │   ├── Navbar.jsx
│       │   ├── VideoPlayer.jsx                <- Video.js wrapper
│       │   ├── ChatPanel.jsx
│       │   ├── ContentCard.jsx                <- reusable poster card
```

```
│       │   ├── EpisodeList.jsx              <- anime episode selector
│       │   ├── SeasonSelector.jsx           <- TV season/episode selector
│       │   ├── RoomCard.jsx
│       │   ├── CreateRoomModal.jsx
│       │   ├── InviteModal.jsx              <- show invite code
│       │   ├── RDKeyModal.jsx               <- Real-Debrid key setup
│       │   ├── MemberList.jsx
│       │   ├── LoadingSpinner.jsx
│       │   ├── ErrorBoundary.jsx
│       │   └── ReconnectOverlay.jsx         <- shown during WS reconnect
│       ├── api/
│       │   ├── consumet.js                  <- Consumet API calls
│       │   ├── anilist.js                   <- AniList GraphQL calls
│       │   ├── tmdb.js                      <- TMDB REST calls
│       │   └── realdebrid.js                <- Real-Debrid API calls
│       └── utils/
│           ├── formatTime.js                <- seconds -> HH:MM:SS
│           └── streamType.js                <- detect stream type from URL
│
├── server/
│   ├── package.json
│   ├── index.js                             <- Express + Socket.IO + static
│   ├── middleware/
│   │   └── requireAuth.js
│   ├── routes/
│   │   ├── rooms.js
│   │   └── users.js
│   └── socket/
│       └── roomSocket.js
│
└── firebase/
    └── firestore.rules
```

5. Environment Variables

5.1 frontend/.env.example

```
# Firebase
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=

# Backend (Cloud Run URL in production, http://localhost:8080 in dev)
VITE_API_BASE_URL=http://localhost:8080

# Consumet API (your self-hosted Cloud Run URL)
VITE_CONSUMET_API_URL=https://your-consumet-service.run.app

# TMDB (get from https://www.themoviedb.org/settings/api)
VITE_TMDB_API_KEY=

# AniList has no API key — it's public GraphQL
```

5.2 server/.env.example

```
PORT=8080
FIREBASE_PROJECT_ID=
ALLOWED_ORIGINS=http://localhost:5173,https://your-cloudrun-url.run.app
# GOOGLE_APPLICATION_CREDENTIALS=/path/to/serviceAccountKey.json  # local dev only
```

Real-Debrid key
The Real-Debrid API key is stored PER USER in Firestore (users/{uid}.rdApiKey). It is NEVER stored in environment variables. Each user enters their own key in their profile settings. The frontend calls Real-Debrid directly from the browser using the user's own key — it never goes through your backend.

6. External Services Setup

6.1 Firebase — same as previous PRD

- Enable Authentication: Email/Password + Google.
- Enable Firestore in production mode, region: asia-southeast1.
- Register web app and copy config to frontend .env.

6.2 TMDB API Key

1. Go to https://www.themoviedb.org/signup and create a free account.
2. Go to Settings > API > Create > Developer.
3. Copy the API Read Access Token (v4 auth). Put it in VITE_TMDB_API_KEY.
4. Free tier: unlimited requests for non-commercial use.

6.3 AniList

No setup needed. AniList GraphQL API is fully public at https://graphql.anilist.co with no authentication required. No API key. Free, unlimited for reasonable use.

6.4 Consumet API — Deploy to Cloud Run

Deploy the official Consumet API as a SEPARATE Cloud Run service:

5. Clone the repo: git clone https://github.com/consumet/api.consumet.org
6. The repo already has a Dockerfile.
7. Build and deploy:

```
gcloud builds submit --tag gcr.io/YOUR_PROJECT/consumet ./api.consumet.org
gcloud run deploy consumet-api \
  --image gcr.io/YOUR_PROJECT/consumet \
  --region asia-southeast1 \
  --allow-unauthenticated \
  --min-instances 0 \
  --max-instances 3 \
  --timeout 60
```

8. Copy the Cloud Run URL to VITE_CONSUMET_API_URL in your frontend .env.

6.5 Real-Debrid

Users must have their own Real-Debrid subscription (~3 EUR/month). They get their API key from https://real-debrid.com/apitoken and enter it in the app's Profile page. The app stores it encrypted in Firestore under users/{uid}.rdApiKey.

7. Database Schema (Firestore)

7.1 Collection: users

Document ID = Firebase Auth UID.

Field	Type	Description
uid	string	Firebase Auth UID
email	string	User email
displayName	string	Display name
photoURL	string|null	Avatar URL
rdApiKey	string|null	Real-Debrid API key (user-entered, stored as-is)
createdAt	Timestamp	Account creation
lastSeen	Timestamp	Last activity

7.2 Collection: watchlist

Document ID = auto. One document per watchlist item per user.

Field	Type	Description
uid	string	Owner UID
contentId	string	AniList ID (anime) or TMDB ID (movie/TV) as string
contentType	string	anime, movie, or tv
title	string	Content title
posterUrl	string	Poster image URL
addedAt	Timestamp	When added to watchlist

7.3 Collection: history

Document ID = uid_contentId (e.g. abc123_12345). Upserted on every watch.

Field	Type	Description
uid	string	User UID
contentId	string	AniList or TMDB ID
contentType	string	anime, movie, or tv

Field	Type	Description
title	string	Content title
posterUrl	string	Poster image URL
episodeId	string|null	Episode ID if anime (e.g. 'naruto-episode-1')
episodeNum	number|null	Episode number
seasonNum	number|null	Season number for TV
position	number	Playback position in seconds
duration	number	Total duration in seconds (0 if unknown)
watchedAt	Timestamp	Last watched time (serverTimestamp)

7.4 Collection: rooms

Same structure as previous PRD plus content fields:

Field	Type	Description
id	string	Document ID
name	string	Room display name
ownerId	string	Creator UID
members	string[]	Member UIDs
hostId	string	Current sync host UID
inviteCode	string	6-char join code
contentId	string|null	What's being watched (AniList/TMDB ID)
contentType	string|null	anime, movie, or tv
contentTitle	string|null	Content title for display
streamUrl	string|null	Current resolved stream URL
episodeId	string|null	Current episode (anime)
playback	map	{ playing, position, updatedAt, updatedBy }
createdAt	Timestamp	Creation time

7.5 Subcollection: rooms/{roomId}/messages

Same as previous PRD: uid, displayName, photoURL, text, createdAt.

8. Content APIs

8.1 Consumet API (Anime Streams)

Base URL: process.env.VITE_CONSUMET_API_URL (your self-hosted instance).

All Consumet calls are made FROM THE FRONTEND directly. They do not go through your backend.

Create frontend/src/api/consumet.js with these functions:

Search anime

```
// GET {CONSUMET}/anime/gogoanime/{query}
export const searchAnime = async (query) => {
  const res = await
fetch(`${CONSUMET}/anime/gogoanime/${encodeURIComponent(query)}`);
  return res.json(); // { results: [{ id, title, url, image, releaseDate,
subOrDub }] }
};
```

Get anime info + episode list

```
// GET {CONSUMET}/anime/gogoanime/info/{animeId}
export const getAnimeInfo = async (animeId) => {
  const res = await fetch(`${CONSUMET}/anime/gogoanime/info/${animeId}`);
  return res.json();
  // returns: { id, title, image, description, status, totalEpisodes,
  //            episodes: [{ id, number, url }] }
};
```

Get streaming sources for an episode

```
// GET {CONSUMET}/anime/gogoanime/watch/{episodeId}
export const getEpisodeSources = async (episodeId) => {
  const res = await
fetch(`${CONSUMET}/anime/gogoanime/watch/${encodeURIComponent(episodeId)}`);
  return res.json();
  // returns: { sources: [{ url, quality, isM3U8 }], subtitles: [...] }
};
```

Source selection logic — pick the best quality available:

```
const pickSource = (sources) => {
  const preferred = ['1080p', '720p', '480p', '360p', 'default', 'backup'];
```

```
  for (const quality of preferred) {
    const src = sources.find(s => s.quality === quality);
    if (src) return src;
  }
  return sources[0]; // fallback to first
};
```

8.2 AniList API (Anime Metadata)

Base URL: https://graphql.anilist.co — no API key, no auth header needed.

Create frontend/src/api/anilist.js with these functions:

Search anime

```
export const searchAniList = async (query, page=1, perPage=20) => {
  const gql = `query($search:String,$page:Int,$perPage:Int){
    Page(page:$page,perPage:$perPage){
      media(search:$search,type:ANIME,sort:POPULARITY_DESC){
        id title{romaji english} coverImage{large} averageScore
        episodes status description genres bannerImage
      }
    }
  }`;
  const res = await fetch('https://graphql.anilist.co', {
    method: 'POST', headers: {'Content-Type':'application/json'},
    body: JSON.stringify({ query: gql, variables: { search: query, page,
perPage } })
  });
  const data = await res.json();
  return data.data.Page.media;
};
```

Get trending anime

```
export const getTrendingAnime = async (page=1, perPage=20) => {
  // same query but without search, sort: TRENDING_DESC
};
```

Get anime by ID

```
export const getAnimeById = async (id) => {
  // query Media(id:$id,type:ANIME) with full fields
  // returns full metadata including studios, trailer, relations
};
```

AniList ID to Consumet ID mapping: AniList returns numeric IDs (e.g. 20). Consumet uses slug IDs (e.g. 'one-piece'). Use this endpoint to resolve:

```
// GET {CONSUMET}/utils/anilist-to-gogoanime?id={anilistId}
export const resolveAnilistToGogoanime = async (anilistId) => {
  const res = await fetch(`${CONSUMET}/utils/anilist-to-
gogoanime?id=${anilistId}`);
  return res.json(); // { id: 'one-piece', title: 'One Piece' }
};
```

8.3 TMDB API (Movies & TV Metadata)

Base URL: https://api.themoviedb.org/3

Auth: Authorization: Bearer VITE_TMDB_API_KEY header on every request.

Create frontend/src/api/tmdb.js:

Function	Endpoint	Returns
searchContent(query, type)	GET /search/{type}?query=	Paginated results (type = movie or tv)
getTrending(type, window)	GET /trending/{type}/{window}	Trending content (window = day or week)
getMovieDetail(id)	GET /movie/{id}?append_to_response=credits,videos	Full movie metadata
getTVDetail(id)	GET /tv/{id}?append_to_response=credits,videos	Full TV show metadata
getTVSeason(id, season)	GET /tv/{id}/season/{season}	Episode list for a season
getGenres(type)	GET /genre/{type}/list	Genre list for movie or tv
discoverContent(type, params)	GET /discover/{type}?...	Filtered browse results

Image URL construction: TMDB images need a base URL. Use:

```
export const tmdbImage = (path, size='w500') =>
  path ? `https://image.tmdb.org/t/p/${size}${path}` : '/placeholder.png';
```

8.4 Real-Debrid API (Movie/TV Streams)

Base URL: https://api.real-debrid.com/rest/1.0

Auth: Authorization: Bearer {user's rdApiKey} on every request.

All calls are made FROM THE FRONTEND using the user's own key stored in their profile.

Create frontend/src/api/realdebrid.js:

Full flow to get a stream URL from a torrent hash

9. Add magnet to Real-Debrid:

```
// POST /torrents/addMagnet
export const addMagnet = async (magnet, rdKey) => {
  const form = new FormData();
  form.append('magnet', magnet);
  const res = await fetch('https://api.real-
debrid.com/rest/1.0/torrents/addMagnet', {
    method: 'POST', headers: { Authorization: `Bearer ${rdKey}` }, body: form
  });
  return res.json(); // { id: 'TORRENT_ID', uri: '...' }
};
```

10. Select files from torrent (auto-select all video files):

```
// POST /torrents/selectFiles/{id}
export const selectFiles = async (torrentId, rdKey) => {
  const form = new FormData();
  form.append('files', 'all');
  await fetch(`https://api.real-
debrid.com/rest/1.0/torrents/selectFiles/${torrentId}`, {
    method: 'POST', headers: { Authorization: `Bearer ${rdKey}` }, body: form
  });
};
```

11. Poll torrent info until status is 'downloaded':

```
// GET /torrents/info/{id}
export const getTorrentInfo = async (torrentId, rdKey) => {
  const res = await fetch(`https://api.real-
debrid.com/rest/1.0/torrents/info/${torrentId}`, {
    headers: { Authorization: `Bearer ${rdKey}` }
  });
  return res.json(); // { status, links: ['https://real-debrid.com/d/...'] }
};
```

12. Unrestrict the link to get a direct stream URL:

```
// POST /unrestrict/link
export const unrestrictLink = async (link, rdKey) => {
  const form = new FormData();
  form.append('link', link);
  const res = await fetch('https://api.real-
debrid.com/rest/1.0/unrestrict/link', {
    method: 'POST', headers: { Authorization: `Bearer ${rdKey}` }, body: form
  });
  return res.json(); // { download: 'https://...' } <- direct stream URL
};
```

Helper function that wraps the full flow:

```
export const resolveStream = async (magnet, rdKey, onProgress) => {
  onProgress?.('Adding torrent...');
  const { id } = await addMagnet(magnet, rdKey);
  await selectFiles(id, rdKey);
  // Poll until downloaded (max 60s)
  for (let i = 0; i < 30; i++) {
    await new Promise(r => setTimeout(r, 2000));
    const info = await getTorrentInfo(id, rdKey);
    onProgress?.(`Resolving... ${info.progress}%`);
    if (info.status === 'downloaded' && info.links?.length > 0) {
      const { download } = await unrestrictLink(info.links[0], rdKey);
      return download; // direct MP4/MKV URL
    }
    if (['error','dead','magnet_error'].includes(info.status)) {
      throw new Error(`Torrent failed: ${info.status}`);
    }
  }
  throw new Error('Timeout: torrent took too long to resolve');
};
```

Where do magnet links come from?
For movies/TV, the user provides a magnet link OR the app can integrate with Torrentio (a public Stremio addon API). Torrentio endpoint: https://torrentio.strem.fun/stream/movie/{imdbId}.json — returns a list of stream sources including magnet links. TMDB provides the IMDB ID via the external_ids endpoint: GET /movie/{id}/external_ids.

9. Feature Specifications

9.1 Authentication (AuthPage.jsx)

Identical to previous PRD (Section 8.1). Route: /auth. Three modes: Login, Sign Up, Forgot Password. Firebase Auth email+password and Google OAuth. On signup, create Firestore users document. Redirect to /home on success.

9.2 Home / Discovery Page (HomePage.jsx)

Route: /home — protected. The landing page after login.

Layout (top to bottom):

- Hero banner: large featured content (first trending anime or movie). Shows title, description snippet, two buttons: 'Watch Now' and 'More Info'.
- Section: 'Continue Watching' — horizontal scroll row of ContentCards from user's Firestore history, ordered by watchedAt desc. Only shown if history is not empty.
- Section: 'Trending Anime' — horizontal scroll row, data from AniList getTrendingAnime().
- Section: 'Trending Movies' — horizontal scroll row, data from TMDB getTrending('movie', 'week').
- Section: 'Trending TV Shows' — horizontal scroll row, data from TMDB getTrending('tv', 'week').

ContentCard component (used everywhere):

- Props: { id, title, posterUrl, contentType, rating, year }
- Shows: poster image, title (truncated), rating badge, contentType badge (ANIME / MOVIE / TV).
- Hover state: slight scale(1.04) + show a play icon overlay.
- Click: navigates to /anime/{id}, /movie/{id}, or /tv/{id} depending on contentType.
- Width: 160px fixed. Height: 240px (poster). Use object-fit: cover.

9.3 Anime Browse & Search (AnimeBrowsePage.jsx)

Route: /anime — protected.

- Top: search bar. On input (debounced 400ms), call AniList searchAniList() and display results in a grid.
- Default state (no search): show genre filter pills (fetched from AniList genres list) and a grid of trending/popular anime.

- Genre filter: clicking a genre chip calls AniList discoverAnime with that genre filter.
- Grid: responsive — 2 cols mobile, 4 cols desktop — of ContentCard components.
- Infinite scroll or 'Load More' button: load next page of results.

9.4 Anime Detail Page (AnimeDetailPage.jsx)

Route: /anime/:anilistId — protected.

Load sequence:

13. Fetch AniList metadata by ID (getAnimeById). Show title, banner image, poster, description, genres, rating, status, episode count.
14. Call resolveAnilistToGogoanime(anilistId) to get the Consumet/GogoAnime ID.
15. Call getAnimeInfo(gogoanimeId) to get the episode list.
16. Render EpisodeList component with the episodes.

Buttons:

- '+ Watchlist': toggle add/remove from Firestore watchlist collection.
- 'Watch in Room': opens CreateRoomModal pre-filled with this content.
- Each episode row: 'Watch' button — navigates to /watch?type=anime&episodeId={id}&animeId={anilistId}.

9.5 Movies & TV Browse/Detail

MovieBrowsePage.jsx — Route: /movies (and /tv for TV shows) — same pattern as AnimeBrowsePage but calls TMDB API.

MovieDetailPage.jsx — Route: /movie/:tmdbId or /tv/:tmdbId:

- Fetch TMDB movie/TV detail with credits and videos.
- For TV: show SeasonSelector component — dropdown of seasons, then episode grid for selected season.
- Show trailer embed if available (YouTube iframe from TMDB videos).
- 'Watch' button: for movies, navigates to /watch?type=movie&tmdbId={id}. For TV episodes: /watch?type=tv&tmdbId={id}&season={s}&episode={e}.
- 'Watch in Room': opens CreateRoomModal.

9.6 Watch Page (WatchPage.jsx)

Route: /watch — protected. This is the core page. Handles both solo watching and room watching.

URL params (query string):

Param	Values	Required
type	anime, movie, tv	Yes
episodeId	Consumet episode ID string	Only for anime
animeId	AniList numeric ID	Only for anime
tmdbId	TMDB numeric ID	Only for movie/tv
season	Season number	Only for tv
episode	Episode number	Only for tv
roomId	Firestore room ID	No (if present, room mode is activated)

Stream resolution on page load:

17. If type=anime: call getEpisodeSources(episodeId) from Consumet. Pick best quality source using pickSource(). If isM3U8=true, stream type is HLS; else MP4.
18. If type=movie or tv: check if user has rdApiKey in their profile. If not, show RDKeyModal asking them to enter their Real-Debrid key. Once key is available, prompt user to enter or paste a magnet link for the content (show a text input). Call resolveStream(magnet, rdKey) with a progress overlay. On success, use the direct URL.
19. Pass the resolved URL and stream type to VideoPlayer component.

Layout:

- If no roomId: full-width video player, no chat panel. Show 'Start Watch Party' button which opens CreateRoomModal.
- If roomId present: same layout as previous PRD — video 70%, chat + members 30%.

Continue watching — save progress:

```
// Save to Firestore history every 10 seconds while playing
// and on pause/unload
const saveProgress = async () => {
  const docId = `${user.uid}_${contentId}`;
  await setDoc(doc(db, 'history', docId), {
    uid: user.uid, contentId, contentType, title, posterUrl,
    episodeId: episodeId || null, episodeNum: episodeNum || null,
    seasonNum: seasonNum || null,
```

```
    position: Math.floor(player.currentTime()),
    duration: Math.floor(player.duration()) || 0,
    watchedAt: serverTimestamp(),
  }, { merge: true });
};
```

9.7 Watch Party Rooms (RoomsPage.jsx)

Route: /rooms — protected. Same as previous PRD RoomCard list. Two differences:

- CreateRoomModal now has a 'Content' field: user can search for anime/movies inline. When content is selected, contentId, contentType, contentTitle are stored in the room.
- Clicking a room card navigates to /watch?type={contentType}&...&roomId={roomId} so the watch page loads with room sync active.

9.8 User Profile (ProfilePage.jsx)

Route: /profile — protected.

- Display name update (same as previous PRD).
- Real-Debrid API key section: shows masked key if saved, 'Update Key' button opens RDKeyModal. Key is saved to Firestore users/{uid}.rdApiKey.
- Watchlist tab: grid of ContentCards from Firestore watchlist collection.
- History tab: list of recently watched items with position (e.g. 'EP 5 • 14:32'). Click resumes playback.
- Logout button.

10. API Endpoints (Backend)

Only room management and user profile go through the backend. All content API calls (Consumet, AniList, TMDB, Real-Debrid) are made directly from the frontend browser.

Method	Path	Auth	Description
GET	/health	None	Health check. Returns { status: 'ok', uptime: N }
GET	/api/me	Required	Get caller's Firestore user profile
PATCH	/api/me	Required	Update displayName or rdApiKey in Firestore
GET	/api/rooms	Required	List rooms where caller is a member
POST	/api/rooms	Required	Create room. Body: { name, contentId, contentType, contentTitle, streamUrl }
GET	/api/rooms/:roomId	Required	Get single room (must be member)
PATCH	/api/rooms/:roomId	Required (owner)	Update room stream URL or content
DELETE	/api/rooms/:roomId	Required (owner)	Delete room + all messages
POST	/api/rooms/join	Required	Join by invite code. Body: { inviteCode }
PATCH	/api/rooms/:roomId/host	Required (owner)	Transfer host to caller
DELETE	/api/rooms/:roomId/members/:uid	Required (owner or self)	Remove member

11. WebSocket Events & Reconnect Strategy

11.1 Socket.IO Events

Identical event set to previous PRD: join-room, leave-room, user-joined, user-left, playback:play, playback:pause, playback:seek, sync:request, sync:state, sync:apply, error.

See previous PRD Section 10 for full payload definitions. All apply here.

11.2 Reconnect Strategy (min-instances=0)

With min-instances=0, Cloud Run instances shut down after ~5 minutes of no HTTP traffic. When the instance restarts (cold start ~3-5 seconds), all WebSocket connections drop. The frontend MUST handle this gracefully.

useSocket.js — Full implementation with reconnect:

```
import { useEffect, useRef, useState } from 'react';
import { io } from 'socket.io-client';

export function useSocket(apiBaseUrl, token) {
  const socketRef = useRef(null);
  const [connected, setConnected] = useState(false);
  const [reconnecting, setReconnecting] = useState(false);

  useEffect(() => {
    if (!token || !apiBaseUrl) return;

    const socket = io(apiBaseUrl, {
      auth: { token },
      transports: ['websocket', 'polling'],  // polling fallback
      reconnection: true,
      reconnectionAttempts: Infinity,        // keep trying forever
      reconnectionDelay: 1000,               // start at 1s
      reconnectionDelayMax: 10000,           // max 10s between attempts
      randomizationFactor: 0.5,
      timeout: 20000,                        // 20s connection timeout
    });

    socket.on('connect', () => {
      setConnected(true);
      setReconnecting(false);
```

```
    });

    socket.on('disconnect', (reason) => {
      setConnected(false);
      // transport close = server went away (cold start)
      if (reason === 'transport close' || reason === 'transport error') {
        setReconnecting(true);
      }
    });

    socket.on('connect_error', () => setReconnecting(true));

    socketRef.current = socket;
    return () => socket.disconnect();
  }, [token, apiBaseUrl]);

  return { socketRef, connected, reconnecting };
}
```

Re-join room after reconnect:

In RoomPage/WatchPage, when the socket reconnects, automatically re-emit join-room:

```
useEffect(() => {
  if (connected && roomId && user) {
    socketRef.current?.emit('join-room', {
      roomId, uid: user.uid, displayName: user.displayName
    });
  }
}, [connected, roomId]);
```

ReconnectOverlay component:

Show this overlay when reconnecting=true. It must NOT block the video player — show it as a banner at the top of the page:

```
export default function ReconnectOverlay({ reconnecting }) {
  if (!reconnecting) return null;
  return (
    <div className='fixed top-14 left-0 right-0 z-50 bg-amber-900/90 text-
amber-200
                    text-sm text-center py-2 flex items-center justify-center
gap-2'>
      <div className='w-3 h-3 border border-amber-400 border-t-transparent
                      rounded-full animate-spin' />
```

```
      Reconnecting to watch party...
    </div>
  );
}
```

Keep-alive HTTP ping
To reduce cold starts, the frontend should ping GET /health every 4 minutes while a user is logged in and on a watch or room page. This keeps the Cloud Run instance warm without paying for min-instances=1. Add this in a useEffect in WatchPage: setInterval(() => fetch('/health'), 4 * 60 * 1000).

12. UI/UX Design Specifications

12.1 Color Palette

Token	Tailwind	Usage
Page background	bg-zinc-950 (#09090b)	All page backgrounds
Surface	bg-zinc-900 (#18181b)	Cards, panels, navbar
Surface raised	bg-zinc-800 (#27272a)	Inputs, hover states, dropdowns
Border	border-zinc-700 (#3f3f46)	Card and input borders
Border subtle	border-zinc-800 (#27272a)	Section dividers
Text primary	text-zinc-50 (#fafafa)	Headings, main text
Text secondary	text-zinc-400 (#a1a1aa)	Labels, descriptions
Text muted	text-zinc-500 (#71717a)	Timestamps, hints
Accent purple	bg-violet-600 (#7c3aed)	Primary buttons, active states
Accent teal	bg-teal-500 (#14b8a6)	ANIME badges, secondary accents
Accent blue	bg-blue-600 (#2563eb)	MOVIE badges
Accent orange	bg-orange-500 (#f97316)	TV badges
Warning	bg-amber-900 text-amber-200	Reconnect banner
Error	bg-red-950 text-red-300	Error messages

12.2 Typography

- Font: 'DM Sans' from Google Fonts (same as previous PRD).
- Set in tailwind.config.js fontFamily.sans.

12.3 Navbar

- Height: 56px. Background: bg-zinc-900 border-b border-zinc-800. Fixed at top, z-index 40.
- Left: logo (video icon + 'StreamTogether').
- Center (desktop only): nav links — Home, Anime, Movies, TV Shows, Rooms.
- Right: user avatar dropdown (Profile, Logout). On mobile: hamburger menu.

12.4 Hero Banner

- Height: 500px on desktop, 300px on mobile.
- Background: banner image from AniList/TMDB (full-width, object-fit cover) with a gradient overlay: from-transparent to-zinc-950 at the bottom.
- Content sits at the bottom-left over the gradient.

12.5 Content Rows (horizontal scroll)

- Container: overflow-x-auto with hidden scrollbar (scrollbar-hide Tailwind plugin or custom CSS: ::-webkit-scrollbar { display: none }).
- Items: flex gap-3, no wrap.
- Each ContentCard: w-40 flex-shrink-0.

12.6 Video Player Styling

- Override Video.js default skin. Add to index.css:

```
.video-js { background: #000; border-radius: 0; }
.vjs-control-bar { background: linear-gradient(transparent, rgba(0,0,0,0.8));
}
.vjs-play-progress { background: #7c3aed; }
.vjs-load-progress { background: rgba(124,58,237,0.3); }
```

13. Page-by-Page Component Specs

13.1 App.jsx — Complete Route Structure

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';

// Routes:
// /                   -> redirect to /home
// /auth               -> AuthPage (public)
// /home               -> HomePage (protected)
// /anime              -> AnimeBrowsePage (protected)
// /anime/:anilistId   -> AnimeDetailPage (protected)
// /movies             -> MovieBrowsePage (protected, type=movie)
// /tv                 -> MovieBrowsePage (protected, type=tv)
// /movie/:tmdbId      -> MovieDetailPage (protected, type=movie)
// /tv/:tmdbId         -> MovieDetailPage (protected, type=tv)
// /watch              -> WatchPage (protected, params via query string)
// /rooms              -> RoomsPage (protected)
// /profile            -> ProfilePage (protected)
// *                   -> redirect to /home
```

13.2 EpisodeList.jsx

Props: { episodes: [{ id, number, url }], currentEpisodeId, onSelect, animeId }

- Renders a scrollable list of episode rows.
- Each row: episode number, 'EP {n}' label, 'Watch' button.
- Current episode row is highlighted with violet border.
- If more than 100 episodes, add a range selector at the top (1-100, 101-200, etc.) to avoid rendering all at once.

13.3 SeasonSelector.jsx

Props: { seasons, selectedSeason, onSeasonChange, episodes, onEpisodeSelect }

- Dropdown/tab list of seasons.
- Below: grid of episode cards showing episode number, name, still image (if available from TMDB), air date.

13.4 RDKeyModal.jsx

A modal that shows when a user tries to watch a movie/TV show without a Real-Debrid key.

- Explains what Real-Debrid is and why it's needed (one sentence).
- Link to https://real-debrid.com to sign up.
- Input for the API key (from https://real-debrid.com/apitoken).
- 'Save Key' button: saves to Firestore users/{uid}.rdApiKey, closes modal.

13.5 CreateRoomModal.jsx

Fields: Room name (required), Content search (optional — inline search bar that queries AniList+TMDB), Private/Public toggle.

On submit: POST /api/rooms. On success: navigate to /watch?...&roomId={id}.

13.6 InviteModal.jsx

Shows when user clicks 'Invite Friends' in a room. Displays:

- The 6-char invite code in large monospace text.
- 'Copy Code' button (copies to clipboard).
- A shareable link: {appDomain}/join/{inviteCode} — clicking this link auto-fills the join dialog.

Handle /join/:code route in App.jsx: redirect to /rooms and auto-open the join dialog with the code pre-filled.

14. Deployment (Cloud Run)

14.1 Dockerfile (main app)

```
FROM node:20-alpine AS frontend-builder
WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
ARG VITE_FIREBASE_API_KEY
ARG VITE_FIREBASE_AUTH_DOMAIN
ARG VITE_FIREBASE_PROJECT_ID
ARG VITE_FIREBASE_STORAGE_BUCKET
ARG VITE_FIREBASE_MESSAGING_SENDER_ID
ARG VITE_FIREBASE_APP_ID
ARG VITE_API_BASE_URL
ARG VITE_CONSUMET_API_URL
ARG VITE_TMDB_API_KEY
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
COPY server/package*.json ./
RUN npm ci --omit=dev
COPY server/ ./
COPY --from=frontend-builder /app/frontend/dist ./public
ENV PORT=8080
EXPOSE 8080
CMD ["node", "index.js"]
```

14.2 Deploy main app

```
gcloud builds submit --tag gcr.io/YOUR_PROJECT/streamtogether \
  --build-arg VITE_FIREBASE_API_KEY=... \
  --build-arg VITE_FIREBASE_AUTH_DOMAIN=... \
  --build-arg VITE_FIREBASE_PROJECT_ID=... \
  --build-arg VITE_FIREBASE_STORAGE_BUCKET=... \
  --build-arg VITE_FIREBASE_MESSAGING_SENDER_ID=... \
  --build-arg VITE_FIREBASE_APP_ID=... \
  --build-arg VITE_API_BASE_URL=https://YOUR_MAIN_CLOUDRUN_URL \
  --build-arg VITE_CONSUMET_API_URL=https://YOUR_CONSUMET_CLOUDRUN_URL \
  --build-arg VITE_TMDB_API_KEY=...
```

```
gcloud run deploy streamtogether \
  --image gcr.io/YOUR_PROJECT/streamtogether \
  --region asia-southeast1 \
  --platform managed \
  --allow-unauthenticated \
  --min-instances 0 \
  --max-instances 5 \
  --timeout 3600 \
  --set-env-vars FIREBASE_PROJECT_ID=...,ALLOWED_ORIGINS=https://YOUR_URL
```

14.3 server/package.json

```
{ "name": "streamtogether-server", "version": "1.0.0", "type": "module",
  "scripts": { "start": "node index.js", "dev": "node --watch index.js" },
  "dependencies": { "cors": "^2.8.5", "express": "^4.18.2",
    "firebase-admin": "^12.0.0", "socket.io": "^4.7.2" } }
```

14.4 frontend/package.json — key dependencies

```
{ "dependencies": { "firebase": "^10.8.0", "react": "^18.2.0",
  "react-dom": "^18.2.0", "react-router-dom": "^6.22.0",
  "socket.io-client": "^4.7.2", "video.js": "^8.6.0" },
  "devDependencies": { "@vitejs/plugin-react": "^4.2.1",
  "tailwindcss": "^3.4.1", "vite": "^5.1.3" } }
```

15. Cold Start & Reconnect Handling

This section is critical for min-instances=0. Every scenario must be handled.

Scenario	User sees	Code behaviour
Page load, instance cold	LoadingSpinner while API warms up	Socket.IO polls with exponential backoff. HTTP requests retry once after 3s delay.
Instance shuts down mid-watch (solo)	Nothing — no WebSocket needed for solo	No impact. Video stream is direct from Consumet/RD to browser.
Instance shuts down mid-room	ReconnectOverlay banner appears	Socket.IO auto-reconnects. On connect, re-emits join-room. Server re-syncs position.
Consumet instance cold (anime stream)	Loading overlay: 'Resolving stream...'	getEpisodeSources() called with a 10s timeout and 2 retries. Show spinner during wait.
Real-Debrid resolution in progress	Progress overlay: 'Resolving 45%...'	resolveStream() polls and calls onProgress callback to update UI.
User joins room, host is offline	Toast: 'Host is not connected. Playback may be out of sync.'	Server checks if hostId socket is in the room. If not, emits warning to new joiner.
Keep-alive ping fails (server down)	No visible impact if not in room	Ping failure is silently swallowed — it's best-effort only.

16. Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    match /watchlist/{docId} {
      allow read, write: if request.auth != null
        && request.auth.uid == resource.data.uid;
      allow create: if request.auth != null
        && request.auth.uid == request.resource.data.uid;
    }

    match /history/{docId} {
      allow read, write: if request.auth != null
        && request.auth.uid == resource.data.uid;
      allow create: if request.auth != null
        && request.auth.uid == request.resource.data.uid;
    }

    match /rooms/{roomId} {
      allow read: if request.auth != null
        && request.auth.uid in resource.data.members;
      allow create: if request.auth != null;
      allow update: if request.auth != null
        && request.auth.uid in resource.data.members;
      allow delete: if request.auth != null
        && request.auth.uid == resource.data.ownerId;

      match /messages/{messageId} {
        allow read, create: if request.auth != null
          && request.auth.uid in
          get(/databases/$(database)/documents/rooms/$(roomId)).data.members;
        allow update, delete: if false;
      }
    }
  }
}
```

17. Error Handling & Edge Cases

Scenario	Expected behaviour
Consumet returns empty sources	Show message: 'No streams found for this episode. Try another source.' Add a 'Try Backup Source' button that calls Consumet with ?server=vidstreaming suffix.
AniList rate limit (90 req/min)	Cache AniList responses in sessionStorage for 5 minutes. Key: anilist_{queryHash}.
TMDB API key missing/invalid	Show banner: 'TMDB API key not configured. Movies and TV browsing is unavailable.' Hide movies/TV nav links.
Real-Debrid key invalid	API returns 401. Show toast: 'Your Real-Debrid key is invalid. Please update it in Profile.' Open RDKeyModal.
Torrent not found on Real-Debrid	resolveStream throws. Show: 'Could not resolve this torrent. Try a different source.'
AniList to Consumet ID not found	resolveAnilistToGogoanime returns null. Show: 'Could not find streams for this anime. It may not be available.' Hide episode list.
Video player error (stream broken)	Video.js error event. Show overlay: 'Stream error. The video could not be loaded.' with a Retry button.
User has no Real-Debrid account	Show RDKeyModal with explanation. Also offer fallback: 'Paste a direct video URL instead' text input.
Room invite code not found	POST /api/rooms/join returns 404. Show: 'Invalid invite code. Check and try again.'
Socket joins room but is not a member	Server emits error event. Client shows toast and redirects to /rooms.
Watchlist add fails (Firestore offline)	Show toast: 'Could not update watchlist. Check your connection.'
History save fails	Silently swallow — history is non-critical. Log to console only.

18. Acceptance Criteria

All of the following must pass for v1 to be complete.

18.1 Authentication

- Sign up with email/password works. User document created in Firestore.
- Login with email/password works.
- Login with Google OAuth works.
- Password reset email is sent.
- Logged-out users cannot access /home, /anime/*, /movie/*, /watch, /rooms, /profile.

18.2 Anime

- Home page shows trending anime cards from AniList.
- Anime browse page search returns results within 1 second (after Consumet warms up).
- Anime detail page shows title, poster, description, episode list.
- Clicking 'Watch' on an episode resolves a stream URL from Consumet and plays it in Video.js.
- HLS streams play correctly (no 404 errors in the network tab).
- Adding anime to watchlist persists after page refresh.

18.3 Movies & TV

- Home page shows trending movies and TV from TMDB.
- Movie detail page shows title, poster, description, cast, trailer.
- TV detail page shows season selector and episode list.
- User without Real-Debrid key sees RDKeyModal when trying to watch.
- User with valid Real-Debrid key can paste a magnet link and resolve it to a stream URL.
- Resolved stream URL plays in Video.js.

18.4 Watch Party & Chat

- User can create a room and get a 6-char invite code.
- Second user can join via invite code.
- Host play/pause/seek syncs to all viewers within 1 second.
- New viewer joining active room gets synced to host's position.
- Chat messages appear for all members in real time.
- ReconnectOverlay appears when connection drops and disappears on reconnect.

- After reconnect, room sync is re-established automatically.

18.5 Profile & History

- Continue watching row on home page shows last 5 watched items.
- Clicking a history item resumes from saved position.
- Real-Debrid key can be saved and updated.
- Watchlist tab shows added items correctly.

18.6 Deployment

- docker build completes with no errors.
- GET /health returns { status: 'ok' }.
- React SPA is served at / and all client-side routes return index.html (no 404s on refresh).
- Consumet Cloud Run service responds to /anime/gogoanime/one-piece within 10 seconds.
- App is accessible via HTTPS on Cloud Run URL.

End of PRD — StreamTogether v1.0